```
end

superman = Person.new_from_string("Clark,Kent")
batman = Person.new_from_string("Bruce|Wayne", "|")
puts superman.full_name
puts batman.full_name
```

produces:

```
Clark Kent
Bruce Wayne
```

In this example, the ExtendedNew module is extended into the Person class, and therefore Person.new_from_string is available.

Ruby also provides prepend. The behavior of prepend is the same as include except that a method in a prepended module is executed before a method of the same name in the class. Typically, the method in the prepended module calls super so that the method in the class is also called. Prepending is often used to add logging or other logistical information to classes.

Iterators and the Enumerable Module

The Ruby collection classes (Array, Hash, and so on) support a large number of operations that do various things with the collection: traverse it, sort it, and so on. You may be thinking, "Gee, it'd sure be nice if *my* class could support all these neat-o features, too!"

Well, your classes *can* support all these neat-o features, thanks to the magic of mixins and module Enumerable. All you have to do is write an iterator called each, which returns the elements of your collection in turn. Mix in Enumerable, and suddenly your class supports methods such as map, include?, and find_all?. If the objects in your collection implement meaningful ordering semantics using the spaceship operator <=> method, you'll also get methods such as min, max, and sort.

Composing Modules

Enumerable is a mixin in the Ruby standard library, implementing a bunch of methods in terms of the host class's each method. One of the methods defined by Enumerable is reduce, which we saw previously on page 71. This method applies a function or operation to the first two elements in the collection and then applies the operation to the result of this computation and to the third element, and so on, until all elements in the collection have been used.

Because reduce is made available by Enumerable, we can use it in any class that includes the Enumerable module and defines the method each. Many built-in classes do this.

```
[1, 2, 3, 4, 5].reduce(:+) # => 15
("a".."m").reduce(:+)      # => "abcdefghijklm"
```

We could also define our own class that mixes in Enumerable and hence gets reduce support:

tut_modules/vowel_finder.rb

```
class VowelFinder
  include Enumerable

  def initialize(string)
    @string = string
  end
```